

● WEEK 2 · SESSION 1 · COMPANION E-BOOK

TALK WITH AI.

*Claude Mastery — Speak the Language
— the week you stop chatting with
Claude and start building on it.*

SESSION 1

How Claude thinks — the architecture you configure.

FOR EVERYONE

Written for beginners and professionals at once.

YOU LEAVE WITH

Projects, system prompts, Skills & Connectors — set up.

WELCOME TO WEEK 2

WEEKLY OBJECTIVE

Last week you sent Claude a message. This week you build the operating system that runs your entire AI workflow — Projects, system prompts, Skills, Connectors, Schedules, Routines and Office, all wired into one workspace you use every day.

Identity shift — AI Aware → Claude Native

THIS BOOK · SESSION 1

SESSION 1

HOW CLAUDE THINKS

Context, memory, Projects, system prompts, Skills, Connectors and MCP — the architecture you configure.

HOW EVERY TOPIC IS BUILT

This is your Week 2 companion — read it alongside the two live sessions. It is written for two people at once: last week’s beginner who can now chat with Claude, and the professional ready to build. Skim the plain-English parts; stop on the prompts, the examples, and the build steps.

WHAT	A plain-English definition of the idea.
WHY	Why it matters as you go Claude Native.
WHERE	Where you actually meet it in the Claude app.
WHEN	When to reach for it — and when to ignore it.
HOW	How it works under the hood, still in plain words.

Every topic also carries: advantages and disadvantages, a real example, three professional prompts at three levels, step-by-step build-up and execution, and the odd Fun Fact.

WHAT'S IN SESSION 1

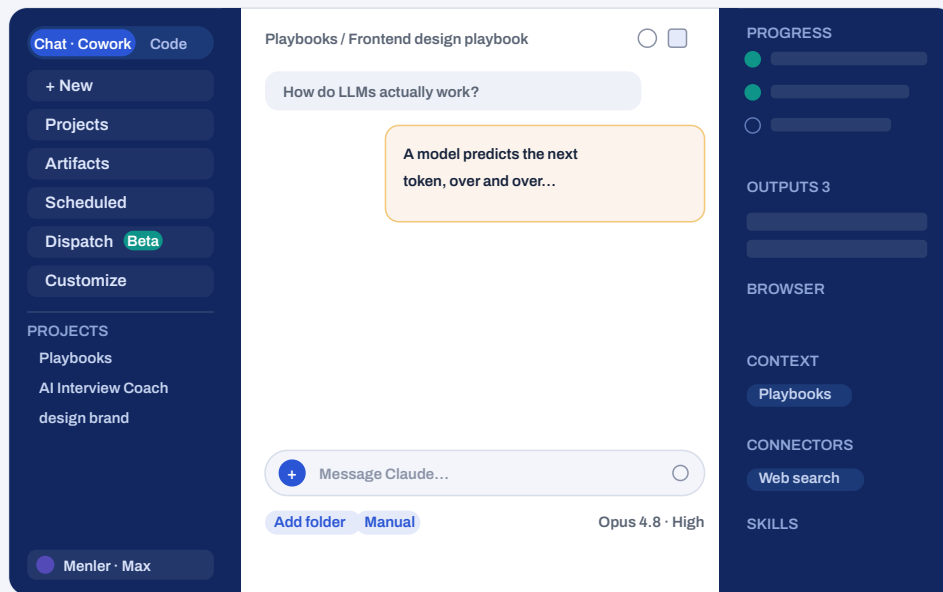
A full tour of the Claude app, two hands-on pages, then the four Session 1 topics — how Claude thinks, and the architecture you configure. Jump to whatever you need.

◆	Getting around Claude The whole interface + how to connect & automate	p. 4
◆	Hands-on: Write a System Prompt A full annotated example you can copy	p. 7
◆	Hands-on: Connect an MCP Wire Claude to any tool via one URL, step by step	p. 8
1.1	Context & Memory Why Claude forgets — and how Projects give it a memory.	p. 9
1.2	Claude Projects Persistent, role-specific environments — Claude that already knows your world.	p. 12
1.3	System Prompts The standing brief Claude reads before every reply — your highest-leverage paragraph.	p. 15
1.4	Skills, Connectors & MCP Extend how Claude behaves, wire it to your data, and reach any tool via one protocol.	p. 18

BEFORE YOU START · THE CLAUDE-FIRST TOUR

GETTING AROUND CLAUDE

This whole program runs on Claude. Learn the app once — here is the entire window, every part labelled.



THE CLAUDE APP — LEFT SIDEBAR, TOP BAR, CHAT (CENTRE), MESSAGE BOX (BOTTOM), AND THE LIVE PANEL (RIGHT)

THE LEFT SIDEBAR — WHERE YOU MOVE BETWEEN KINDS OF WORK

New — Start a fresh chat or Cowork session. Nothing carries over unless the session shares a Project.

Artifacts — Your gallery of built pages and docs (dashboards, mini-apps, reports) that live on beyond the chat and can be re-opened or shared.

Dispatch — Hand a job to a background agent that plans and runs it while you do other things. (Beta.)

Projects list — Your projects pinned for quick access; click one to jump straight into its space.

Projects — Your Projects — shared folders of chats, files and instructions that persist across sessions and your team.

Scheduled — Tasks that run on a schedule — hourly, daily, weekly. View, pause, edit or run them here.

Customize — Your home for connectors, custom instructions, styles and preferences — set once, applies everywhere.

Account · plan — Bottom-left: who you are and your plan (e.g. Max). The download icon gets the desktop app.

THE TOP BAR — WHERE YOU ARE, AND HOW TO GET AROUND

Chat · Cowork / Code — The mode switch. “Chat & Cowork” for everyday work with files and tools; “Code” opens Claude Code, the developer surface.

Back / forward — Move through screens you have visited, like a browser.

Panel & theme — Top-right icons: show or hide the side panels and switch light / dark.

Search — Find any past chat, project or output by keyword.

Breadcrumb — Shows where you are (e.g. Playbooks / Frontend design...). Click a crumb to go up a level.

THE MESSAGE BOX — HOW YOU BRIEF CLAUDE AND CONTROL IT

The “+” — Add connectors, upload files, and switch on skills for this message.

Approval: Manual / Auto / Skip — When Claude asks before acting. Manual = approve every action; Auto = auto-approves reads, checks writes; Skip = no prompts (trusted work only).

Model · reasoning — Pick the model (e.g. Opus 4.8) and the thinking effort — High for hard problems, lower for speed.

Add folder — Give the session a folder from your computer to read and write — pick the smallest folder that holds what is needed.

Dictate — The mic — talk instead of type.

THE PANEL ON THE RIGHT — WHAT CLAUDE SHOWS YOU WHILE IT WORKS

Progress — The live task list — every step Claude is taking, ticking off as it goes.

Browser · Claude in Chrome — The browser Claude is driving for web tasks, when a browser is connected.

Connectors — The apps switched on for this session (e.g. Web search); add more from “+” or Customize.

Outputs — The finished files from this session — open, preview, download, or send some to Drive.

Context — What this session can see — the connected Project and folders (e.g. Playbooks).

Skills — The packaged know-how Claude can apply here on demand.

◆ FIND IT IN CLAUDE**Message box → Manual / Auto / Skip**

Set the approval mode before a big task. New to Cowork? Start on **Manual** so you approve each action. As you trust a routine, move to **Auto**. Keep **Skip** for small, contained jobs only.

THE THREE THINGS THAT TURN CLAUDE FROM A CHATBOT INTO A WORKSPACE

Anyone can type into Claude. The step up is **connecting your own apps**, **putting work on repeat**, and **handing off jobs to the background**. Here is where each lives and how to switch it on.

1

CONNECT YOUR APPS

CONNECTORS

A connector plugs one of your apps — Gmail, Google Drive, Notion, Slack, and hundreds more — straight into Claude, so it can read and act on that data instead of you copy-pasting.

- 1 Open Settings → Connectors (or, in a chat, click “+” → Connectors → Manage connectors).
- 2 Click the “+” next to Connectors to open the directory, then pick the app you want.
- 3 Click Connect, then sign in and approve access in the pop-up (this is OAuth — you approve, Claude never sees your password).
- 4 Back in any chat, click “+” → Connectors and toggle the app on for that conversation.

Free, Pro, Max, Team & Enterprise. Team/Enterprise: an Owner enables a connector for the org first. Choose the least access that does the job.

2

PUT WORK ON REPEAT

SCHEDULED TASKS

A scheduled task is a prompt Claude runs on its own on a set cadence — a Monday digest, a daily inbox triage — and reports back with no nudge from you.

- 1 In the left sidebar, click Scheduled.
- 2 Click New task → Create with Claude (guided) or Set up manually.
- 3 Write the prompt as a full standalone instruction, pick the cadence (hourly / daily / weekly / weekdays), and set the approval mode + folder.
- 4 Click Schedule. Pause, edit, or run it on demand any time from the same page.

Pro, Max, Team & Enterprise. Desktop for all paid plans; web & mobile in beta.

3

HAND OFF, WALK AWAY

DISPATCH

Dispatch is a background agent. You describe an outcome once; it breaks the job into tasks, runs each on its own, and surfaces the results later — even from your phone.

- 1 In the left sidebar, select Dispatch.
- 2 Describe the outcome like you would brief a colleague, in one message.
- 3 Let it plan — it starts child tasks that each show a status (Running, Awaiting input, Completed...).
- 4 Check back later: open any task to read the transcript and grab the files it produced.

Pro & Max, on the latest Claude Desktop (macOS/Windows). Keep the desktop app open to assign from mobile.

HANDS-ON · YOUR HIGHEST-LEVERAGE PARAGRAPH

WRITE A SYSTEM PROMPT

*Four labelled parts — Persona, Context, Rules, Never-do.
Copy this, swap the orange.*

SYSTEM PROMPT · PASTE INTO A PROJECT'S INSTRUCTIONS

PERSONA · CONTEXT · RULES · NEVER-DO

You are my senior research analyst in field.

CONTEXT — Assume I am preparing decisions / reports for a non-technical audience. I value accuracy over speed.

RULES — Default to a 5-bullet answer, each under 20 words.
Lead with the takeaway. Cite the source for every hard fact.
Ask before making assumptions about my data.

NEVER — Never invent statistics or citations. Never use hype words. Never pad; if you are unsure, say so.

Persona — who Claude is here — the role sets the whole tone.

Context — what to assume about you, your work and your audience.

Rules — how to respond — format, length, tone. Be specific and testable.

Never-do — the guardrails — one line per bad habit you want gone.

◆ FIND IT IN CLAUDE

Sidebar → Projects → instructions

Where it goes, and how to test it. Paste this into a **Project's instructions** (or your Custom Instructions for global behaviour). Then run a normal request, change one line, and re-run it — feeling the difference is how you tune it.

HANDS-ON · THE UNIVERSAL ADAPTER

CONNECT AN MCP

A built-in connector is the easy path; a custom MCP connector wires Claude to almost any tool via one URL.

WHAT YOU NEED

One thing: the tool's **remote MCP server URL** — a public `https://` link (often ending `/mcp`) that the provider gives you, or that you deploy for an internal tool. That single URL is what turns “no built-in connector” into “connected”.

1

PRO / MAX — INDIVIDUAL

ADD A REMOTE MCP CONNECTOR

- 1 Go to Customize → Connectors.
- 2 Click the “+”, then “Add custom connector”.
- 3 Paste the tool's **remote MCP server URL** (a public `https://` link, usually ending `/mcp`).
- 4 Optional: add an OAuth Client ID / Secret under Advanced, then click Add.

Free (one), Pro, Max, Team & Enterprise. The URL comes from the tool's provider, or from an MCP server you deploy.

2

TEAM / ENTERPRISE

OWNER ADDS IT, MEMBERS CONNECT

- 1 Owner: Organization settings → Connectors → Add → hover Custom → Web.
- 2 Owner: paste the remote MCP server URL (OAuth in Advanced if needed) → Add.
- 3 Members: Customize → Connectors → find the connector the Owner added.
- 4 Members: click Connect and sign in to authenticate.

On Team/Enterprise an Owner must add a custom MCP connector for the org before members can use it.

3

USE IT

TURN IT ON IN A CHAT

- 1 In any chat, click the “+” in the message box → Connectors.
- 2 Toggle your MCP connector on for the conversation.
- 3 Its tools are now available — ask Claude to use them in plain language.
- 4 Attach it to a Project so every chat there can reach it.

Remote MCP runs from the cloud and needs a public URL. Local MCP (`claude_desktop_config.json`) is Desktop-only and is a separate mechanism — not available in Cowork or `claude.ai`.

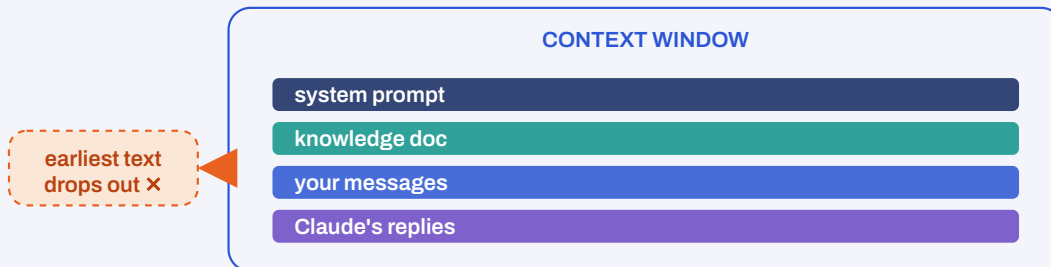
1.1

SESSION 1

CONTEXT & MEMORY

Why Claude forgets — and how Projects give it a memory.

The context window — what Claude can hold at once



a Project re-loads the important parts every new chat

WHAT

Two ideas explain Claude's "memory". The **context window** is everything Claude can hold in view at once — your messages, its replies, uploaded files. **Statelessness** means each new chat starts blank: Claude does not remember yesterday's conversation unless something re-supplies it. **Projects** are what re-supply it — persistent instructions, knowledge and memory that load into every chat inside them.

WHY

This is the single idea that stops the two biggest beginner frustrations: "why did it forget what I said" (you left the window, or started a new chat) and "why do I keep re-explaining myself" (you are not using a Project). Understand it and you stop fighting the tool.

WHERE

Every chat has a context window — a long thread eventually pushes the start out of view. Persistent memory lives in **Projects** (sidebar) and in the built-in memory some surfaces keep across chats.

WHEN

Mind the window on long documents and marathon chats — paste key facts near the top or summarise as you go. Reach for a Project the moment a task is ongoing rather than one-off.

HOW IT WORKS

All the tokens inside the window are processed together each turn; there is no lookup of older, out-of-window text. When you pass the limit, the earliest text silently drops. A Project sidesteps this by re-injecting its system prompt, knowledge documents and memory at the start of every conversation — so the important context is always “in view” again.

ADVANTAGES

- + Modern context windows hold whole books at once
- + Projects give Claude durable, reusable memory
- + Knowing the limit turns confusion into control

DISADVANTAGES

- Past the window, early text is dropped silently
- A plain chat still forgets once it ends
- Stale Project memory can quietly mislead

EXAMPLE

Paste a 60-page report and Claude summarises it in one go — it fits the window. But open a fresh chat tomorrow and ask “what did we decide?” and it draws a blank — unless that chat lives in a Project holding the decision.



Fun Fact

Claude’s context window can hold roughly a couple of full-length novels at once. Yet by default it still starts every brand-new chat with total amnesia — capacity and memory are two different things.

THREE PROFESSIONAL PROMPTS, THREE LEVELS

LEVEL 1 · BEGINNER

FILL THE ORANGE

In plain words, explain what a "context window" is and why you sometimes forget what I said earlier in a long chat. Use one everyday analogy and keep it under 120 words.

LEVEL 2 · INTERMEDIATE

FILL THE ORANGE

I run long research chats and lose earlier detail. Give me 4 habits to keep key facts inside your context window, and tell me the 2 signs that I am near the limit.

LEVEL 3 · PROFESSIONAL

FILL THE ORANGE

Act as a Claude workflow architect. For my ongoing project, design a memory strategy: what belongs in a Project system prompt, what belongs in an uploaded knowledge doc, and what I should paste per-chat. Explain the trade-off for each. Output as a table.

BUILD IT UP — STEP BY STEP

- 1 Open a long chat and paste a big document; ask for a one-paragraph summary.
- 2 Keep chatting until early details start slipping — notice the window at work.
- 3 Now create a Project and put the key facts in its instructions.
- 4 Start a fresh chat inside the Project and ask about those facts — it remembers.

EXECUTE IT

- 1 For long inputs, summarise first, then work from the summary to save the window.
- 2 Keep a reusable "context block" of core facts to paste into new plain chats.
- 3 Move anything you repeat often into a Project's instructions or knowledge docs.
- 4 Refresh Project knowledge as it changes — retire anything out of date.

1.2

SESSION 1

CLAUDE PROJECTS

Persistent, role-specific environments — Claude that already knows your world.

SYSTEM PROMPT

persona · context · constraints · output rules

SKILLS

format · tone · domain rules

KNOWLEDGE DOCS

your notes, SOPs, sources

CONNECTORS

live data from your tools

stacked together = a persistent, role-specific Project

WHAT

A **Project** is Claude with standing context. It bundles a **system prompt** (who Claude is here, and the rules), your **Skills**, uploaded **knowledge documents**, and **connector** access — all shared by every chat inside it. It is the container that turns a general chatbot into a specialist.

WHY

Projects are the backbone of a Claude OS. Instead of re-briefing Claude each time, you configure a role once — “my research analyst”, “my content strategist” — and every chat inside inherits it. This week’s whole build sits on Projects.

WHERE

In the left sidebar under **Projects**. Create one per role, domain or workflow; open it and start chatting with all its context already loaded.

WHEN

Spin one up for any ongoing area — study, a client, a research theme, a content channel. Keep quick, unrelated questions in a plain chat so you do not clutter a Project.

HOW IT WORKS

The system prompt sets persona, context and output rules. Knowledge docs give it your facts. Skills shape the format. Connectors feed live data. At the start of every chat inside the Project, all of this is loaded first — so Claude answers as your configured specialist, not a blank model.

ADVANTAGES

- + Configure a role once; reuse it forever
- + One specialised Claude per domain or client
- + The foundation every automation is built on

DISADVANTAGES

- A weak system prompt makes a weak Project
- Too many overlapping Projects fragments context
- Uploaded knowledge must be kept current

EXAMPLE

A “Thesis” Project: system prompt = “you are my research supervisor in molecular biology”, your papers uploaded, a citation-style Skill attached, and the Notion connector on. Every chat inside already knows your topic, your sources and your format.



Fun Fact

A Project is really just a saved system prompt plus some files and tools — but that small bundle is the difference between talking to Claude and building with Claude.

◆ FIND IT IN CLAUDE

Sidebar → Projects → New project

Build the Project in the sidebar. Create the Project, paste your system prompt into its instructions, attach a Skill, upload a knowledge doc, and enable a Connector — every chat inside then inherits it. See the annotated system prompt on p.7.

THREE PROFESSIONAL PROMPTS, THREE LEVELS

LEVEL 1 · BEGINNER

FILL THE ORANGE

I want to set up a Claude Project for my main use case.
In simple terms, tell me what to put in it and give me a short starter system prompt I can paste in.

LEVEL 2 · INTERMEDIATE

FILL THE ORANGE

Help me plan 3 Claude Projects for my life: one for work, one for study/research, one for creative. For each, give the persona, what knowledge to upload, and one Skill to attach. Put it in a table.

LEVEL 3 · PROFESSIONAL

FILL THE ORANGE

Act as my Claude architect. Design a Project for my role end to end: the full system prompt, the Skills to attach, the knowledge docs to upload, the Connectors to enable, and 3 tasks it should handle without me re-briefing it. Output as a build checklist.

BUILD IT UP — STEP BY STEP

- 1 In the sidebar, create a Project and name it after a real role you play.
- 2 Write its system prompt: persona, context, output format, and what to never do.
- 3 Upload one knowledge document that Claude should always know.
- 4 Attach a Skill and connect one tool (e.g. Notion).

EXECUTE IT

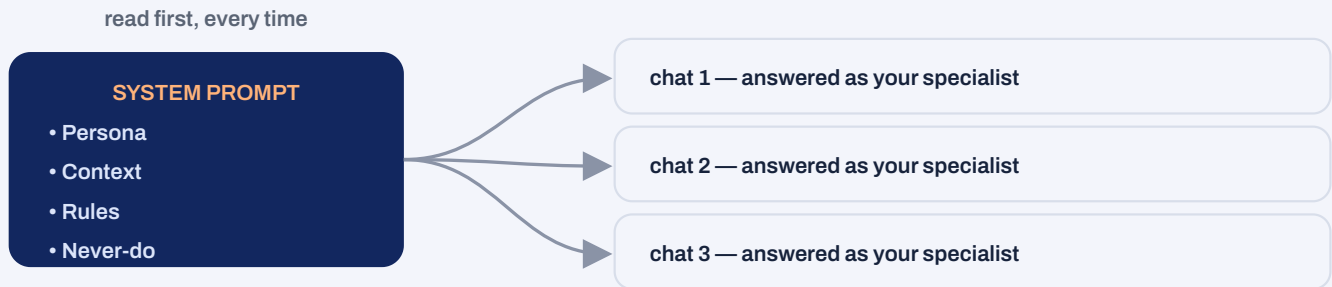
- 1 Run 3 real tasks inside the Project without re-briefing — note what it nails.
- 2 Tighten the system prompt wherever the output drifts.
- 3 Keep one Project per role; resist dumping everything into one.
- 4 Revisit and refresh its knowledge docs every few weeks.

1.3

SESSION 1

SYSTEM PROMPTS

The standing brief Claude reads before every reply — your highest-leverage paragraph.

**WHAT**

A **system prompt** is the instruction Claude reads *before* your message — it sets who Claude is, what it should assume, how it should respond, and what it must never do. In a Project it applies to every chat; it is the most powerful configuration layer you touch.

WHY

A sharp 150-word system prompt out-performs hours of clever one-off prompting, because you set the rules once and they apply to everything. It is the difference between a helpful stranger and a colleague who already knows the brief.

WHERE

In a Project's **instructions** field, and in Custom Instructions / your profile for global behaviour. The hands-on page (p.7) shows a full annotated example.

WHEN

Write one for every Project, and update it whenever the output drifts. For a one-off chat you rarely need a system prompt — a good message is enough.

HOW IT WORKS

Claude treats the system prompt as higher-priority standing context: it is loaded first and colours every reply. The winning structure is **persona** (who you are) + **context** (what to assume) + **rules** (how to respond) + **guardrails** (what to never do). Specific beats clever — name the format, the tone and the audience.

ADVANTAGES

- + Set the rules once; they apply to everything
- + Turns Claude into a reliable, on-brand specialist
- + Easy to version and improve over time

DISADVANTAGES

- Vague system prompts give vague, generic output
- Over-stuffing one can box Claude in — leave room to think
- Contradictory rules confuse more than they help

EXAMPLE

Compare “help me write” with a system prompt that says “You are my B2B copywriter for CFOs; default to 90-word LinkedIn posts in a plain, confident tone; always end with one question; never use hype words.” Same model, completely different, on-brand output every time.



Fun Fact

The word “system” in system prompt is literal: Claude reads it the way a computer reads its boot instructions — before anything else, and it frames everything that follows.

THREE PROFESSIONAL PROMPTS, THREE LEVELS

LEVEL 1 · BEGINNER

FILL THE ORANGE

Write me a simple system prompt for a Claude Project about my role. Cover who you are here, what to assume about my work, and how to format answers. Keep it clear and under 120 words.

LEVEL 2 · INTERMEDIATE

FILL THE ORANGE

Write a 150-word Project system prompt for my use case with four labelled parts: Persona, Context, Rules, and Never-do. Make the rules specific about format, length and tone.

LEVEL 3 · PROFESSIONAL

FILL THE ORANGE

Act as a prompt engineer. Interview me with 5 questions about my role, then draft a production-grade system prompt from my answers, plus a one-line note on how to test whether it is working. Leave bracketed slots for anything you need from me.

BUILD IT UP — STEP BY STEP

- 1 Pick one Project and open its instructions field.
- 2 Draft four labelled parts: Persona, Context, Rules, Never-do.
- 3 Make each rule concrete — name the format, length, tone and audience.
- 4 Paste it in and run a normal request to see it take effect.

EXECUTE IT

- 1 Change one line and re-run the same request to feel the difference.
- 2 Add a “never-do” line for each bad habit you notice.
- 3 Keep a copy of each system prompt so you can version and improve it.
- 4 Re-read it monthly — prune anything that no longer fits.

1.4

SESSION 1

SKILLS, CONNECTORS & MCP

Extend how Claude behaves, wire it to your data, and reach any tool via one protocol.

**WHAT**

Three layers extend Claude. **Skills** change how it behaves (reusable rule sets, called with /). **Connectors** wire it to your apps — Notion, Drive, Slack, Gmail — for live, bidirectional access. **MCP** (Model Context Protocol) is the universal standard that lets Claude connect to *any* external tool, and the **API** lets Claude run as code, not just chat.

WHY

Prompting alone hits a ceiling. Skills make output consistent, Connectors give Claude your real context, and MCP means almost anything with an MCP server — internal tools included — can plug in. This is what “Claude Native” actually means.

WHERE

Skills and Connectors live under **Customize** and the message-box “+”; MCP connectors are added the same way (or configured for teams); the API lives in code and the developer console.

WHEN

Skill it when you repeat an output type. Connect it when a task needs your data. Reach for MCP/API when you need a tool with no built-in connector, or want Claude embedded in software.

HOW IT WORKS

A Skill is a SKILL.md rule set Claude applies on top of your prompt. A Connector is an OAuth-scoped bridge to one app. MCP standardises those bridges so one protocol serves many tools — think of it as a universal adapter. The API exposes the same model to your own code for automation at scale.

ADVANTAGES

- + One protocol (MCP) connects Claude to almost anything
- + Skills + Connectors together beat prompting alone
- + The API turns Claude into infrastructure, not just chat

DISADVANTAGES

- More power means more setup and permissions to manage
- Every connection widens what must be kept secure
- MCP/API depth can need a developer for custom tools

EXAMPLE

Call **/meeting-notes** (a Skill) on a transcript pulled live from your **Notion** (a Connector), inside a Project — no copy-paste. Need a tool with no built-in connector? An **MCP** server bridges it; the **API** runs the whole thing on a schedule from code.



Fun Fact

MCP has been called “the USB-C of AI tools” — one standard plug so any tool can talk to any model, instead of a tangle of one-off integrations.

◆ FIND IT IN CLAUDE

Customize → Connectors → Add custom connector

Connect a custom tool via MCP. Built-in connectors turn on from **Customize** → **Connectors**. For a tool with no built-in connector, add its **remote MCP server URL** as a custom connector — the full step-by-step is on p.8.

THREE PROFESSIONAL PROMPTS, THREE LEVELS

LEVEL 1 · BEGINNER

FILL THE ORANGE

Explain the difference between a Claude Skill and a Claude Connector in plain words, with one example of each for [my work]. Under 120 words.

LEVEL 2 · INTERMEDIATE

FILL THE ORANGE

For my role, recommend 2 Skills to build and 3 Connectors to turn on, with a one-line reason for each and the first task I should run once they are set up.

LEVEL 3 · PROFESSIONAL

FILL THE ORANGE

Act as a solutions architect. Map my workflow onto Skills, Connectors, MCP and API: which layer handles each part, where a custom MCP or the API is justified over a built-in connector, and the security check for each connection. Output as a table.

BUILD IT UP — STEP BY STEP

- 1 Turn on one Connector (Notion or Drive) under Customize → Connectors.
- 2 Build or attach one Skill for an output you produce often.
- 3 Run a task that uses both together inside a Project.
- 4 Note one tool you use that would need an MCP connector or the API.

EXECUTE IT

- 1 Grant the least access each Connector needs; review before approving.
- 2 Keep Skills single-purpose so they compose cleanly in a Project.
- 3 Shortlist MCP/API needs and flag which truly need a developer.
- 4 Document each connection so a teammate could rebuild it.

SESSION 1 DONE

YOU KNOW HOW CLAUDE THINKS.

Context and memory, Projects, system prompts, Skills, Connectors and MCP — the architecture of your Claude workspace is now clear.

Next up: Session 2 — Schedules, Routines, Plugins and Claude for Office, then your Milestone build.

AI Aware → Claude Native

NEXT · SESSION 2

Session 1 was the architecture. **Session 2** is the automation — putting work on a timer, packaging it as Plugins and Routines, and generating Office files. Continue in the Session 2 book, where you also build Milestone Project 1.

menler

Your turning point in the AI era.